

Practical 4

COS214

Event Name:

TaskForge Logistics Domain

Group Members:

Maeryn Pillay- u25146484

Kristy Heesen- u24702341

Livhuwani Munyai- u25449029

TASK 1

Event Description:

TaskForge is a corporate facilities complaint and maintenance management system designed for companies that operate one or more buildings or sites. The system allows employees, tenants or facility staff to log and manage complaints relating to building maintenance, equipment, cleaning, safety, access control and other facility-related issues. Complaints are organised hierarchically by site, department or floor, complaint category and individual complaint.

Each complaint follows a defined lifecycle consisting of Logged, Assigned, In Progress, Resolved and Closed. The behaviour of a complaint depends on its current state, ensuring that only valid actions and transitions can occur.

The system supports two main traversal methods. A complete traversal visits all complaints in the hierarchy, while an unresolved traversal visits only complaints that have not yet reached the Resolved or Closed states. Complaints are processed according to the date and time at which they were logged, with older complaints receiving attention first.

Complaints are normally processed in the order in which they are logged. An Urgent Complaint decorator can be applied dynamically to give a complaint higher processing priority, causing urgent complaints to be processed before normal complaints while still preserving first-logged, first-processed ordering among complaints of the same priority. A Notification decorator can also be applied dynamically to add notification behaviour when important actions occur on a complaint. These decorators can be stacked on the same complaint without changing the underlying complaint class.

Go4 Participant Identification:

Composite

Component - ComplaintComponent

Leaf - Complaint

Composite - ComplaintGroup

Concrete Composites - Company, Building, FloorDepartment, ComplaintCategory

Client - main.cpp

Iterator

Iterator - ComplaintIterator

Concrete Iterators - AllIterator, UnresolvedIterator

Aggregate - ComplaintGroup

Client - main.cpp

State

Context - Complaint

State - ComplaintState

Concrete States - LoggedState, AssignedState, InProgressState, ResolvedState, ClosedState

Decorator

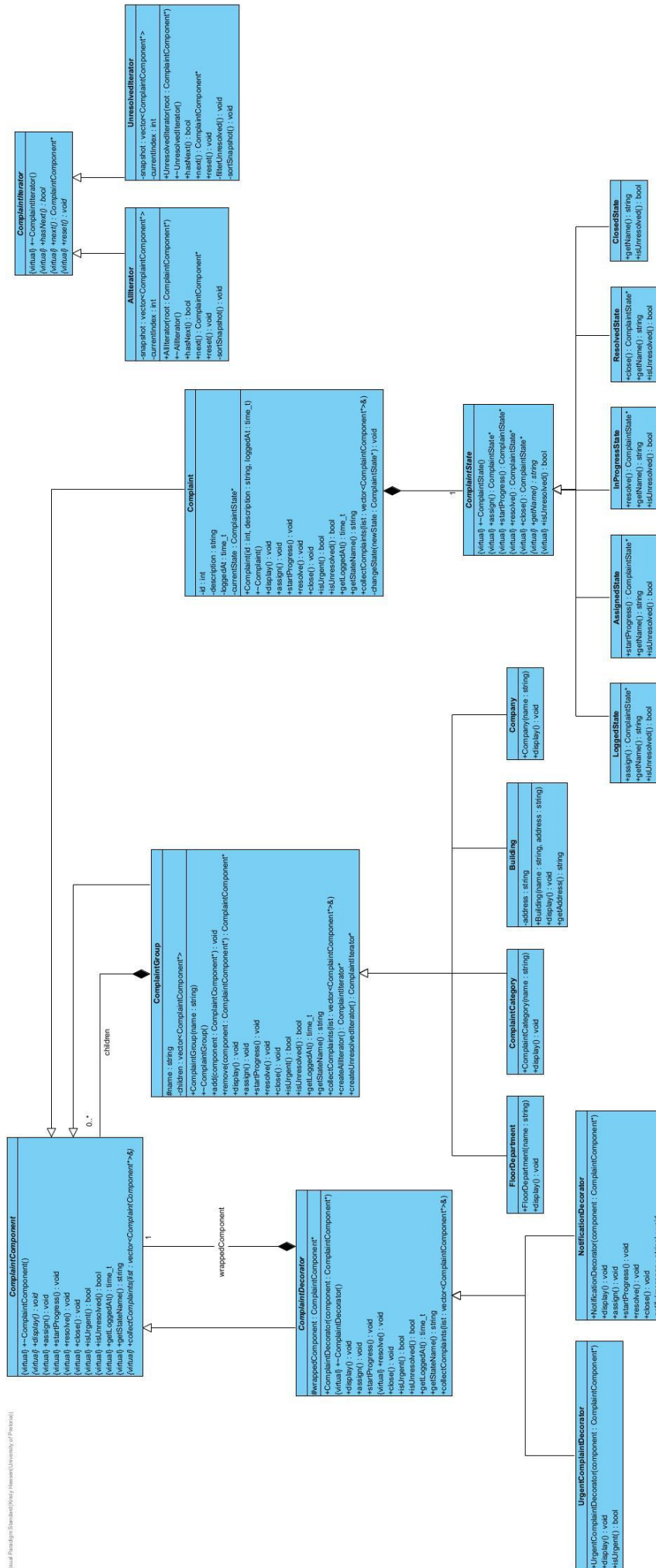
Component - ComplaintComponent

Concrete Component - Complaint

Decorator - ComplaintDecorator

Concrete Decorators - UrgentComplaintDecorator, NotificationDecorator

Client - main.cpp



Inheritance, Associations, Multiplicities and Ownership:

ComplaintComponent provides the common abstraction used throughout the complaint hierarchy. Complaint represents an individual leaf, while ComplaintGroup represents a composite capable of containing other ComplaintComponent objects. Company, Building, FloorDepartment and ComplaintCategory specialise ComplaintGroup and form the nested hierarchy of the system.

A ComplaintGroup owns zero or more ComplaintComponent children, represented by a composition relationship with multiplicity 0..*. This ownership allows the root company to recursively destroy the complete hierarchy when it is deleted.

Each Complaint owns exactly one ComplaintState at a time. When a valid state transition occurs, the existing state is replaced by the next state. This relationship is represented as composition with multiplicity 1.

A ComplaintDecorator owns exactly one wrapped ComplaintComponent. Concrete decorators may be stacked, allowing an urgent complaint to also contain notification behaviour. Deleting the outer decorator recursively destroys the wrapped decorator or complaint.

The iterators do not own the complaints they traverse. They maintain their own traversal snapshot and position, allowing two iterators to traverse the same hierarchy independently without transferring ownership of any complaints.

Concrete Domain Classes:

The design contains concrete domain classes representing the organisation of a corporate facilities complaint system. A Company contains one or more Building objects, which may contain FloorDepartment objects. These contain ComplaintCategory groups, which finally contain individual Complaint objects. This creates a meaningful recursive hierarchy with both nested groups and individual work items.

Individual complaints progress through the concrete lifecycle states LoggedState, AssignedState, InProgressState, ResolvedState and ClosedState. UrgentComplaintDecorator and NotificationDecorator provide additional runtime behaviour without requiring separate complaint subclasses.

Design and Ownership Rationale:

The Composite pattern was selected because complaints naturally belong to a nested organisational structure. Using a common ComplaintComponent abstraction allows both individual complaints and complete groups to be treated uniformly while keeping the internal child container hidden from client code.

The State pattern separates complaint lifecycle behaviour from the Complaint class. A complaint progresses through Logged, Assigned, In Progress, Resolved and Closed states, with each state controlling the valid transition to the next stage. Invalid transitions are handled by the current state rather than through a large conditional statement.

The Decorator pattern allows additional responsibilities to be applied dynamically. A normal complaint is represented by an undecorated Complaint. Applying an UrgentComplaintDecorator gives it higher processing priority, while a NotificationDecorator adds notification behaviour. The decorators may be stacked while the underlying complaint remains unchanged.

The Iterator pattern prevents client code from accessing the internal containers of the hierarchy directly. AllIterator traverses all complaints, while UnresolvedIterator returns only

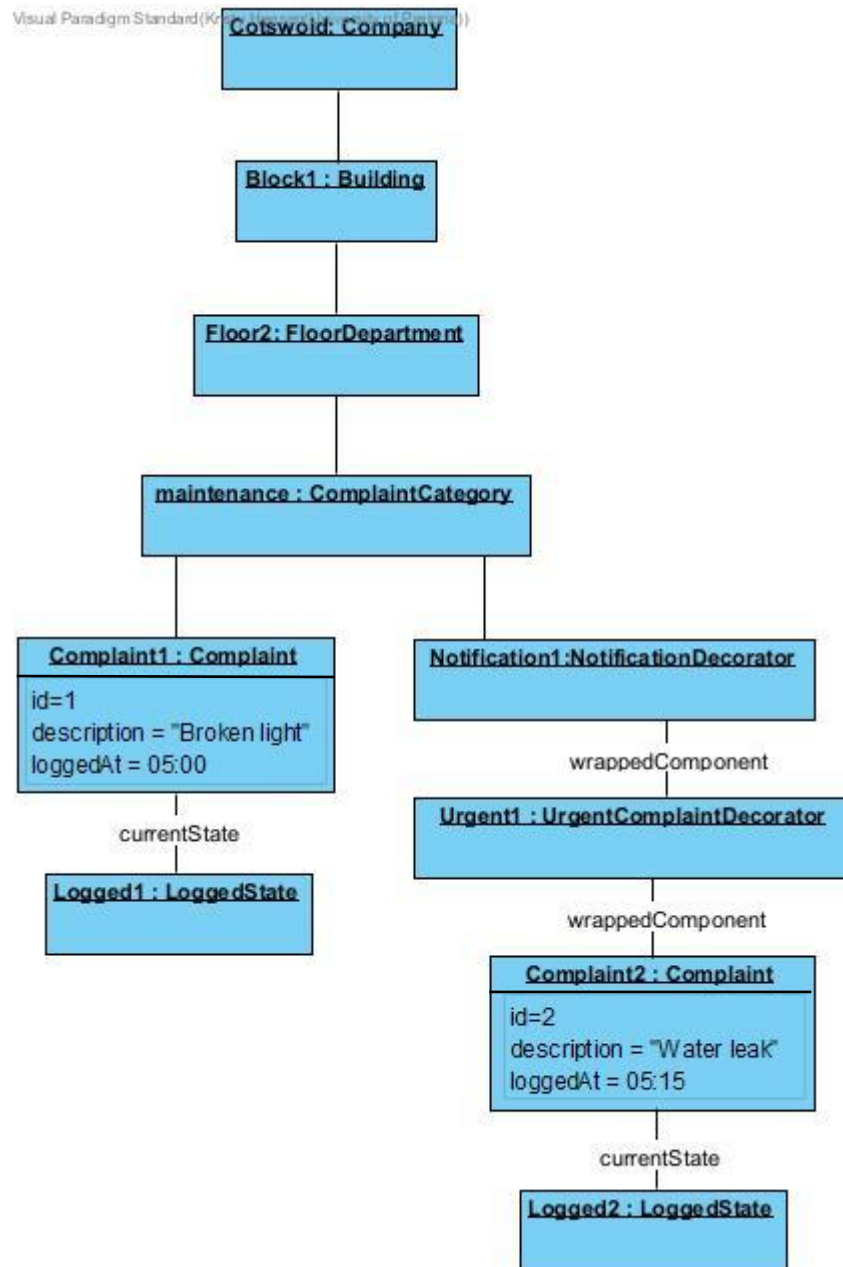
complaints that have not reached the Resolved or Closed states. Both traversals prioritise urgent complaints before normal complaints. Within each priority group, complaints are processed from oldest to newest according to the time they were logged.

Each iterator maintains its own snapshot and current position, allowing multiple independent traversals of the same hierarchy. If new complaints are added after an iterator has been created, they are not included in that existing snapshot and a new iterator must be created to see them. Moving an existing complaint between groups does not affect an existing snapshot because the complaint object itself remains alive. Objects are not deleted while an iterator containing them is active.

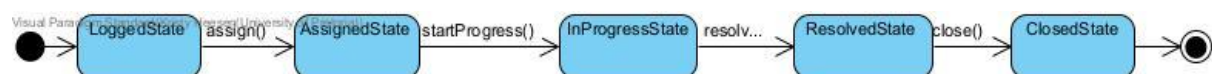
Ownership is kept hierarchical. `main.cpp` owns the root `Company`, complaint groups own their children, complaints own their current state, and decorators own their wrapped component. This provides a clear destruction path and allows the complete system to be cleaned up without memory leaks.

TASK 4

Object Diagram:



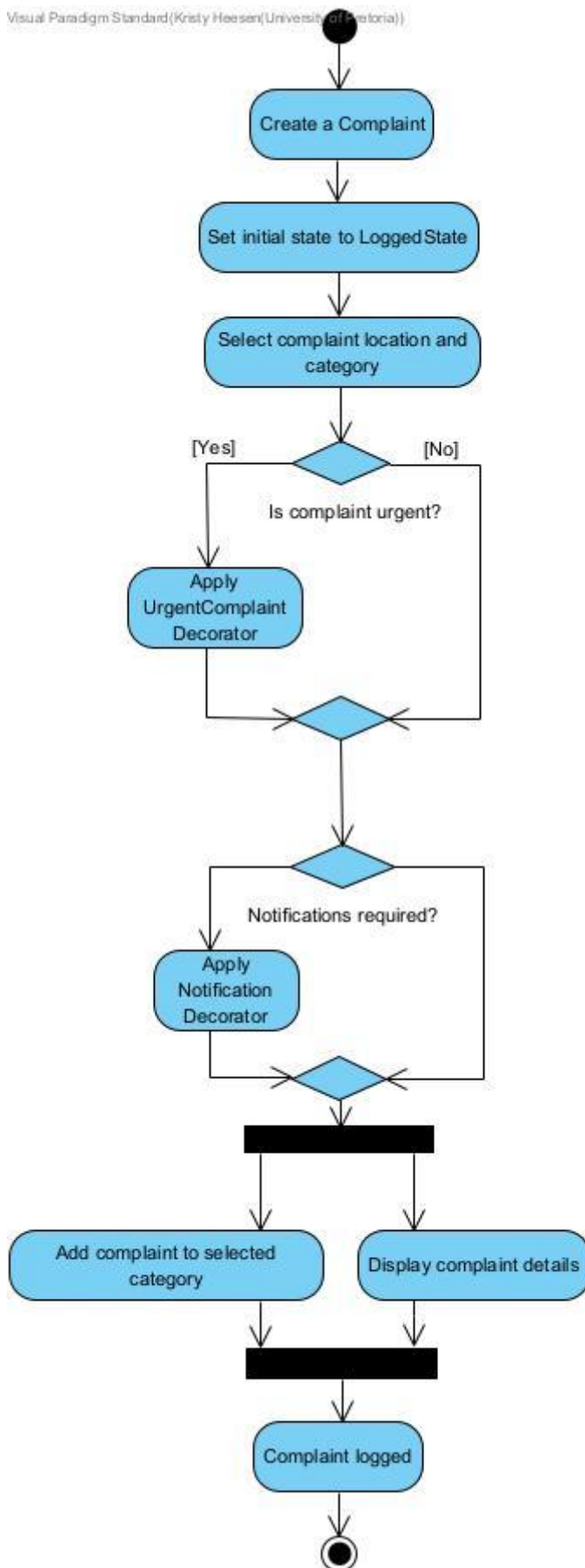
State Diagram:



Activity Diagram 1 - Log and prioritise a complaint:

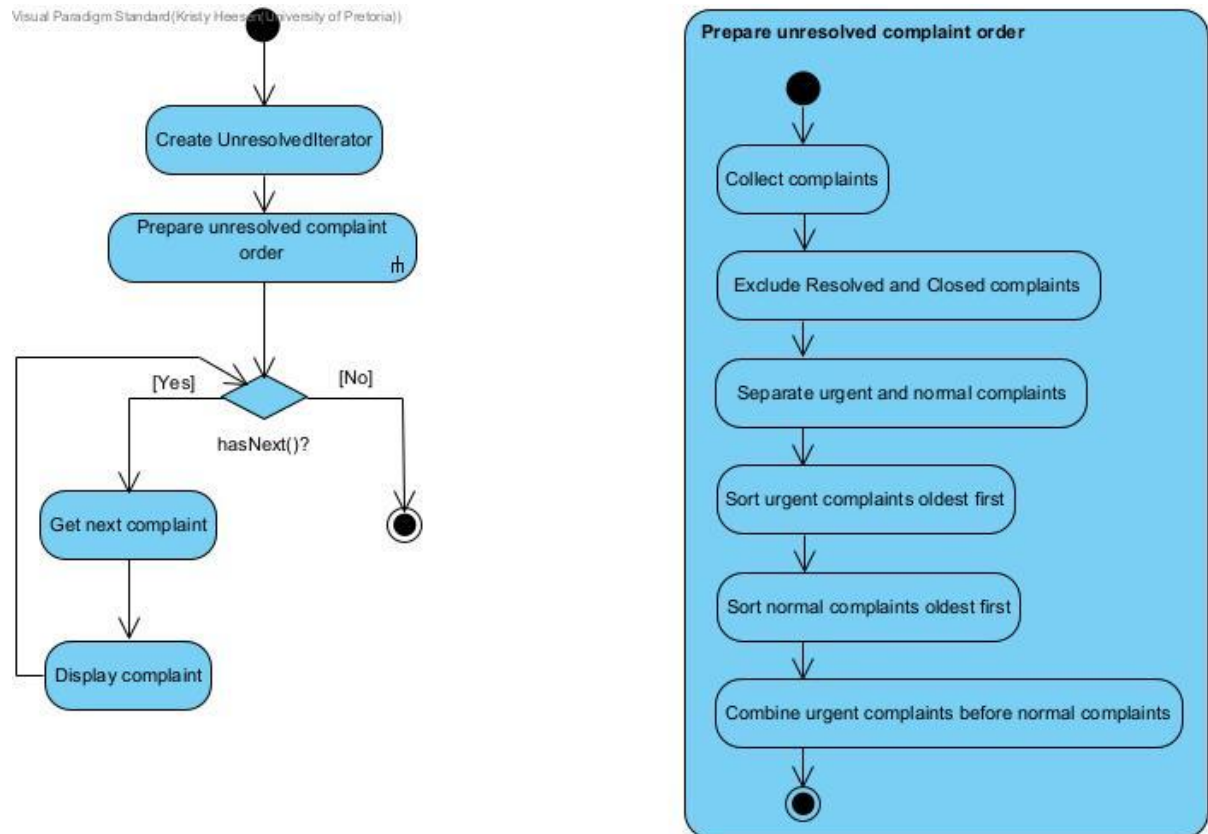
Visual Paradigm Standard (Kristy Heesen (University of Pretoria))

- Decisions
- Guards
- Merges
- Fork/join

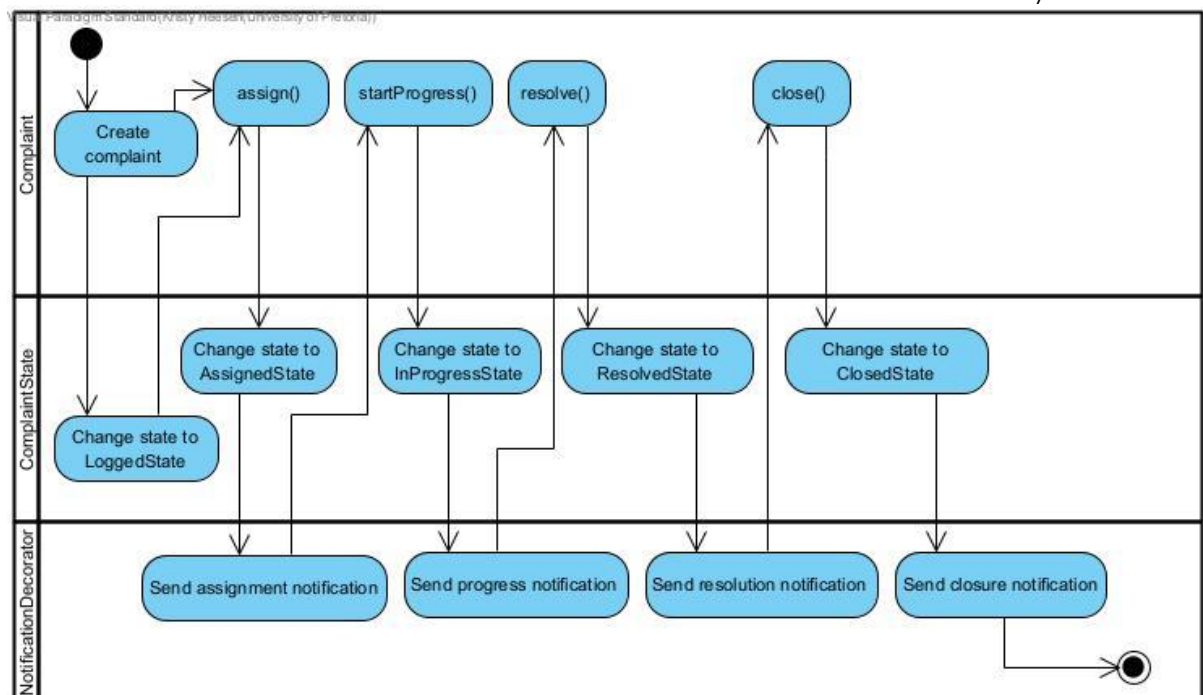


Activity Diagram 2 - Traverse unresolved complaints:

- loop
- called/composite activity
- visible traversal

**Activity Diagram 3 - Process a complaint through its lifecycle:**

- Swim lanes
- lifecycle workflow



TASK 5

GDB Evidence:

```

u25146484@Lament: /mnt/c/Users/maery/OneDrive/Documents/Uni/2nd Year/S2/COS214/Practicals/Prac_4/214-Prac-4-group49$ gdb ./taskforge
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2015 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./taskforge...
(gdb) break main
Breakpoint 1 at 0x5988: file main.cpp, line 170.
(gdb) run
Starting program: /mnt/c/Users/maery/OneDrive/Documents/Uni/2nd Year/S2/COS214/Practicals/Prac_4/214-Prac-4-group49/taskforge

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at main.cpp:170
170      runScenario1();
(gdb) step
runScenario1 () at main.cpp:44
44      void runScenario1() {
(gdb) n
      std::cout << "=====\\n";
(gdb) n
=====
46      std::cout << " SCENARIO 1: Escalation Pipeline & Lifecycle Processing\\n";
(gdb) n
SCENARIO 1: Escalation Pipeline & Lifecycle Processing
47      std::cout << "=====\\n\\n";
(gdb) n
=====
49      Company* company = new Company("TaskForge Logistics HQ");
(gdb) n
      Building* mainBuilding = new Building("Main Distribution Hub", "100 Logistics Way");
50
(gdb) print *company
$1 = {<ComplaintGroup> = {<ComplaintComponent> = {_vptr.ComplaintComponent = 0x555555568708 <vtable for Company+16>, name = "TaskForge Logistics HQ",
      children = std::vector of length 0, capacity 0}, <No data fields>}
(gdb) quit
A debugging session is active.

        Inferior 1 [process 630] will be killed.

Quit anyway? (y or n) y
u25146484@Lament: /mnt/c/Users/maery/OneDrive/Documents/Uni/2nd Year/S2/COS214/Practicals/Prac_4/214-Prac-4-group49$

```

Bug Encountered During Development:

While testing the Iterator traversal, complaints that were wrapped in decorators were not displaying their decorated behaviours. They were missing the [URGENT] tag in the output and were also not being prioritised at the top of the list by the UnresolvedIterator. The problem was in the ComplaintDecorator::collectComplaints() function. The decorator was delegating the collection down to its child by calling wrappedComponent->collectComplaints(list). This meant the raw, undecorated base Complaint object pushed itself into the Iterator's snapshot, which basically ended up removing the decorator wrappers during the traversal. It appeared that there was no decorator.

```

SCENARIO 1: Escalation Pipeline & Lifecycle Processing

--- Initial System Hierarchy ---
Group: TaskForge Logistics HQ
Group: Main Distribution Hub
Group: Floor 1 - Operations
Group: Equipment Maintenance
Complaint #101: Light bulb replacement [Logged]
[NOTIFICATION SENT] [URGENT]Complaint #102: Main Conveyor Belt Stoppage [Logged]
[NOTIFICATION SENT] Complaint #103: Water Leak in restroom [Logged]

--- Traversing All Complaints (Priority Ordered via AllIterator) ---

Breakpoint 1, ComplaintDecorator::collectComplaints (this=0x55555557cb40, list=std::vector of length 1, capacity 1 = {...}) at ComplaintDecorator.cpp:52
52      if(wrappedComponent) wrappedComponent->collectComplaints(list);
(gdb) print this
$1 = (ComplaintDecorator *) 0x55555557cb40
(gdb) print wrappedComponent
$2 = (ComplaintComponent *) 0x55555557ca00
(gdb) step

Breakpoint 1, ComplaintDecorator::collectComplaints (this=0x55555557ca00, list=std::vector of length 1, capacity 1 = {...}) at ComplaintDecorator.cpp:52
52      if(wrappedComponent) wrappedComponent->collectComplaints(list);

```

In the GDB screenshot, we paused the code right when the iterator tries to collect a decorated complaint. First we printed this to look at the outer wrapper. Then we printed `wrappedComponent` to see the plain complaint hiding inside it. When we used the `step` command to move to the next line of code, GDB jumped straight into the inner complaint instead of stopping at the wrapper. This proved that the code was accidentally throwing away the decorator wrapper and only handing the plain, undecorated complaint over to the iterator.

To fix this we changed a single line of code in `ComplaintDecorator.cpp`. Instead of having the wrapper tell the inner complaint to add itself to the list, we changed the code to simply say `list.push_back(this)`. This forced the decorator wrapper to add itself to the iterator's list instead. Now when the iterator prints or sorts the complaints, it correctly sees the wrappers and applies the [URGENT] tags like it is supposed to.

Valgrind Evidence:

```

u25146484@Lament: /mnt/c/Users/maery/OneDrive/Documents/Uni/2nd Year/S2/COS214/Practicals/Prac_4/214-Prac-4-group49$ make valgrind
valgrind \
  --leak-check=full \
  --show-leak-kinds=all \
  --track-origins=yes \
  --error-exitcode=1 \
  ./taskforge
==669== Memcheck, a memory error detector
==669== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==669== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==669== Command: ./taskforge
==669==

=====
SCENARIO 1: Escalation Pipeline & Lifecycle Processing
=====

--- Initial System Hierarchy ---
Group: TaskForge Logistics HQ
Group: Main Distribution Hub
Group: Floor 1 - Operations
Group: Equipment Maintenance
Complaint #101: Light bulb replacement [Logged]
[NOTIFICATION SENT] [URGENT]Complaint #102: Main Conveyor Belt Stoppage [Logged]
[NOTIFICATION SENT] Complaint #103: Water leak in restroom [Logged]

--- Traversing All Complaints (Priority Ordered via AllIterator) ---
[NOTIFICATION SENT] [URGENT]Complaint #102: Main Conveyor Belt Stoppage [Logged]
Complaint #101: Light bulb replacement [Logged]
[NOTIFICATION SENT] Complaint #103: Water leak in restroom [Logged]

--- Lifecycle State Transitions ---
Assigning Complaint #102...
Notification: Complaint has been assigned
Starting progress on Complaint #102...
Notification: Complaint has been started
Resolving and closing Complaint #102...
Notification: Complaint has been resolved
Notification: Complaint has been closed

--- Traversing Unresolved Snapshot (Captured Before Resolution) ---
Complaint #101: Light bulb replacement [Logged]
[NOTIFICATION SENT] Complaint #103: Water leak in restroom [Logged]

=====
SCENARIO 2: Dynamic Restructuring & Runtime Decoration
=====

--- Initial System Setup ---
Group: TaskForge Logistics Network
Group: Site A - Sorting
Group: Dept A - Inbound
Complaint #104: Hydraulic Fluid Leak [Logged]
Group: Site B - Receiving
Group: Dept B - Outbound

--- Moving Complaint #104 from Dept A to Dept B ---
Complaint #104 successfully re-assigned to Dept B.

--- Dynamically Elevating Complaint #104 with Urgent & Notification Decorators ---

--- Pre-Modification Snapshot Output ---
Complaint #104: Hydraulic Fluid Leak [Logged]

--- Post-Modification Output ---
[NOTIFICATION SENT] [URGENT]Complaint #104: Hydraulic Fluid Leak [Logged]
==669==
==669== HEAP SUMMARY:
==669==   in use at exit: 0 bytes in 0 blocks
==669==   total heap usage: 95 allocs, 95 frees, 77,165 bytes allocated
==669==
==669== All heap blocks were freed -- no leaks are possible
==669==
==669== For lists of detected and suppressed errors, rerun with: -s
==669== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
u25146484@Lament: /mnt/c/Users/maery/OneDrive/Documents/Uni/2nd Year/S2/COS214/Practicals/Prac_4/214-Prac-4-group49$ |

```

TASK 6

GitHub Workflow Reflection:

The team used GitHub throughout the development of TaskForge to coordinate work, review changes and integrate each member's contribution into the final project. At the beginning of development, a branch ruleset was configured to protect the main branch. Team members could not merge their own work directly into main. Changes were developed on separate feature branches and submitted through pull requests, which required approval from another team member before they could be merged. This ensured that changes were reviewed before becoming part of the final system and reduced the risk of conflicting or incorrect code being added to the main branch.

The practical was divided equally between the three team members. Kristy Heesen was responsible for Task 1, including the system design and UML class diagram, and Task 4, including the object, state and activity diagrams. Kristy also created and configured the GitHub repository and branch protection rules. Maeryn Pillay completed Task 2, implementing the core C++ classes and design patterns, as well as Task 5, which covered debugging and memory investigation. Livhuwani Munyai completed Task 3, including the runtime scenarios and dynamic behaviour, and Task 6, which covered Docker and the GitHub workflow requirements. Although individual tasks were assigned to specific members, the completed work was reviewed and integrated collaboratively.

The repository history demonstrates this workflow. For example, Pull Request #4, "Visual Paradigm UML," contained the initial UML work and was reviewed and approved by Maeryn before being merged. Pull Request #8, "Implementation of .cpp files," contained Maeryn's implementation work and was reviewed and approved by Kristy before integration into main. Pull Request #10 contained work contributed by Livhuwani and was likewise reviewed and approved before being merged. Comments and reviews were also used to confirm that the UML structure, implementation and runtime behaviour remained consistent. This workflow provided a clear development history showing contributions from all three team members while ensuring that the protected main branch contained reviewed and integrated work.

GitHub Repository link:

<https://github.com/Kristy2005/214-Prac-4-group49>

Team Contribution Statement:

The workload was divided equally, with each member responsible for two practical tasks. Kristy Heesen completed Tasks 1 and 4 and created the GitHub repository and protected branch workflow. Maeryn Pillay completed Tasks 2 and 5, while Livhuwani Munyai completed Tasks 3 and 6. Everyone participated in reviewing, approving and integrating one another's work through GitHub pull requests.